Ring of the Middle Earth

Distributed Application Development — Term Project

Weight: 30% of Total Grade
Team Size: Up to 3 students (3 recommended)
Submission: Week 14 — demo and report in a single session

HOW TO READ THIS DOCUMENT

This document has four parts:

- **Part 1 — Game Specification:** Rules, map, units, and combat. The same for everyone.
- **Part 2 — Common: Kafka:** The event infrastructure. Required for both technology choices.
- **Part 3A — Option A: Akka:** Implement the game engine with Akka actors and cluster sharding.
- **Part 3B — Option B: Go:** Implement the game engine with Go goroutines and Kafka state stores.

Read Part 1 and Part 2 fully before writing any code.
Then read only the option you have chosen.

Your architecture document must explain why you chose your option and what the tradeoffs are compared to the other.

PART 1 — GAME SPECIFICATION

---

1. Overview

1.1 What You Are Building

A browser-based, turn-based strategy game backed by a distributed system. Two human players — one controlling the Light Side, one controlling the Dark Side — play in separate browsers. There is no computer-controlled player.

- **Light Side (Free Peoples):** Move the Ring Bearer secretly from The Shire to Mount Doom and destroy the Ring.
- **Dark Side (The Shadow):** Find the Ring Bearer and intercept it before it reaches Mount Doom.

1.2 Win Conditions

Light Side wins when all three are true at the same turn end:

- Ring Bearer is in `mount-doom`
- A `DestroyRing` order was submitted this turn
- No Dark Side unit is present in `mount-doom`

Dark Side wins when both are true at the same turn end:

- Any Nazgul occupies the same region as the Ring Bearer
- The Ring Bearer is `exposed == true` this turn

`exposed` becomes `true` this turn when:

- Any Nazgul is within its effective detection range of the Ring Bearer, OR
- The Ring Bearer crossed a path with `surveillanceLevel >= 1` this turn

`exposed` resets to `false` at the end of every turn.

Draw: After 40 turns with no winner.

1.3 Hidden Start

For the first 3 turns, detection is suppressed. No `RingBearerDetected` or `RingBearerSpotted` event is emitted regardless of Nazgul positions. Detection activates normally from Turn 4 onward.

1.4 Information Asymmetry

The **Light Side player** always sees the Ring Bearer's true position.
The **Dark Side player never sees** the Ring Bearer's position unless a detection event fires.

Both players see the same map, all unit positions, and all path statuses — except the Ring Bearer's location. This asymmetry must be enforced throughout your system.

---

2. The Map

2.1 Regions

22 regions. Fixed. Do not add, remove, or rename any.

regionId	name	terrain	specialRole	startControl	startThreat
the-shire	The Shire	PLAINS	RING_BEARER_START	FREE_PEOPLES	0
bree	Bree	PLAINS	NONE	NEUTRAL	1
tharbad	Tharbad	SWAMP	NONE	NEUTRAL	2
weathertop	Weathertop	MOUNTAINS	NONE	NEUTRAL	2
rivendell	Rivendell	MOUNTAINS	NONE	FREE_PEOPLES	0
fangorn	Fangorn	FOREST	NONE	FREE_PEOPLES	0
fords-of-isen	Fords of Isen	PLAINS	NONE	NEUTRAL	2
rohan-plains	Rohan Plains	PLAINS	NONE	FREE_PEOPLES	1
moria	Moria	MOUNTAINS	NONE	NEUTRAL	3
helms-deep	Helm's Deep	FORTRESS	NONE	FREE_PEOPLES	1
isengard	Isengard	FORTRESS	SHADOW_STRONGHOLD	SHADOW	3
edoras	Edoras	PLAINS	NONE	FREE_PEOPLES	1
lothlorien	Lothlórien	FOREST	NONE	FREE_PEOPLES	0
dead-marshes	Dead Marshes	SWAMP	NONE	NEUTRAL	2
emyn-muil	Emyn Muil	MOUNTAINS	NONE	NEUTRAL	2
minas-tirith	Minas Tirith	FORTRESS	NONE	FREE_PEOPLES	1
ithilien	Ithilien	FOREST	NONE	NEUTRAL	2
osgiliath	Osgiliath	PLAINS	NONE	NEUTRAL	3
minas-morgul	Minas Morgul	FORTRESS	SHADOW_STRONGHOLD	SHADOW	4
cirith-ungol	Cirith Ungol	MOUNTAINS	NONE	SHADOW	4

regionId	name	terrain	specialRole	startControl	startThreat
mordor	Mordor	VOLCANIC	SHADOW_STRONGHOLD	SHADOW	5
mount-doom	Mount Doom	VOLCANIC	RING_DESTRUCTION_SITE	SHADOW	5

Terrain: `PLAINS` `MOUNTAINS` `FOREST` `FORTRESS` `VOLCANIC` `SWAMP`

SpecialRole: `RING_BEARER_START` `RING_DESTRUCTION_SITE` `SHADOW_STRONGHOLD` `NONE`

2.2 Paths

35 paths. All bidirectional. Starting state: `status=OPEN`, `surveillanceLevel=0`, `blockedBy=null`.

`cost` = turns required to traverse.

#	pathId	from	to	cost
1	shire-to-bree	the-shire	bree	1
2	bree-to-weathertop	bree	weathertop	1
3	bree-to-rivendell	bree	rivendell	2
4	bree-to-tharbad	bree	tharbad	1
5	shire-to-tharbad	the-shire	tharbad	2
6	weathertop-to-rivendell	weathertop	rivendell	1
7	rivendell-to-moria	rivendell	moria	2
8	rivendell-to-lothlorien	rivendell	lothlorien	2
9	moria-to-lothlorien	moria	lothlorien	1
10	lothlorien-to-emyn-muil	lothlorien	emyn-muil	1
11	lothlorien-to-rohan-plains	lothlorien	rohan-plains	1
12	rohan-plains-to-fangorn	rohan-plains	fangorn	1
13	rohan-plains-to-edoras	rohan-plains	edoras	1
14	rohan-plains-to-minas-tirith	rohan-plains	minas-tirith	2
15	fangorn-to-isengard	fangorn	isengard	1
16	isengard-to-rohan-plains	isengard	rohan-plains	1
17	tharbad-to-fords-of-isen	tharbad	fords-of-isen	2

#	pathId	from	to	cost
18	fords-of-isen-to-isengard	fords-of-isen	isengard	1
19	fords-of-isen-to-helms-deep	fords-of-isen	helms-deep	1
20	fords-of-isen-to-edoras	fords-of-isen	edoras	1
21	edoras-to-helms-deep	edoras	helms-deep	1
22	helms-deep-to-isengard	helms-deep	isengard	1
23	edoras-to-minas-tirith	edoras	minas-tirith	2
24	emyn-muil-to-dead-marshes	emyn-muil	dead-marshes	1
25	emyn-muil-to-ithilien	emyn-muil	ithilien	2
26	dead-marshes-to-ithilien	dead-marshes	ithilien	1
27	dead-marshes-to-mordor	dead-marshes	mordor	2
28	ithilien-to-minas-tirith	ithilien	minas-tirith	1
29	ithilien-to-osgiliath	ithilien	osgiliath	1
30	ithilien-to-cirith-ungol	ithilien	cirith-ungol	2
31	minas-tirith-to-osgiliath	minas-tirith	osgiliath	1
32	osgiliath-to-minas-morgul	osgiliath	minas-morgul	1
33	minas-morgul-to-cirith-ungol	minas-morgul	cirith-ungol	1
34	minas-morgul-to-mordor	minas-morgul	mordor	1
35	cirith-ungol-to-mordor	cirith-ungol	mordor	1
36	cirith-ungol-to-mount-doom	cirith-ungol	mount-doom	2
37	mordor-to-mount-doom	mordor	mount-doom	1

2.3 Ring Bearer Routes

Four canonical routes. Verify your graph by confirming all four are discoverable via BFS.

Route 1 — Fellowship (13 turns):
the-shire → bree → weathertop → rivendell → moria → lothlorien → emyn-muil → ithilien → cirith-ungol → mount-doom

Route 2 — Northern Bypass (12 turns):
the-shire → bree → rivendell → lothlorien → emyn-muil → dead-marshes → ithilien → cirith-ungol → mount-doom

Route 3 — Dark Route (12 turns):
the-shire → bree → rivendell → lothlorien → emyn-muil → dead-marshes → mordor → mount-doom

Route 4 — Southern Corridor (13 turns):
the-shire → tharbad → fords-of-isen → edoras → minas-tirith → osgiliath → minas-morgul → cirith-ungol → mount-doom

Route 4 bypasses Lothlórien entirely. It passes through Shadow-controlled territory and is the only independent southern corridor.

2.4 Strategic Notes

Two independent corridors: The northern corridor (through Lothlórien) and the southern corridor (through Tharbad and the Fords of Isen). With only 3 Nazgul, the Dark Side cannot permanently block both simultaneously. This is the core strategic tension.

Isengard guards the southern corridor. Saruman can permanently corrupt paths along Route 4. Isengard's destruction disables Saruman.

Path blocking requires presence. A path remains BLOCKED only while the blocking unit stays at one of its endpoint regions. If that unit moves away or is destroyed, the path reverts to OPEN or THREATENED. This rule gives every non-Nazgul unit a meaningful role.

FellowshipGuards protect the Ring Bearer indirectly. A FellowshipGuard stationed at a path endpoint prevents a Nazgul from permanently blocking that path — the Nazgul must defeat the guard first. When FellowshipGuards follow the same route as the Ring Bearer, they occupy the path endpoints ahead and behind, making it significantly harder for the Dark Side to seal off the route. Their combat power is also available to contest any region the Ring Bearer needs to pass through.

---

3. Units

3.1 Design Principle

All units share one implementation class. Behaviour is entirely driven by configuration. No unit name or ID is hardcoded in game logic. Adding a new unit requires only a new config entry — zero code change.

This applies to both Option A and Option B, and it is verified live during the Q&A.

3.2 Unit Classes

```
UnitClass
  ├── RingBearer        — carries the Ring; hidden position; same movement
```

```
as others
  ├── FellowshipGuard   — Light Side combat unit; standard movement
  ├── GondorArmy        — Light Side army; can fortify regions
  ├── Nazgul            — Dark Side wraith; detects Ring Bearer; some
respawn
  ├── UrukHaiLegion     — Dark Side army; ignores fortress terrain bonus
when attacking
  └── Maia              — Powerful ancient being; one special ability per
unit
```

3.3 Unit Configuration

```
 units = [

   # LIGHT SIDE — 7 units

   { id="ring-bearer",   name="Frodo Baggins",
     class="RingBearer", side="FREE_PEOPLES",  start="the-shire",
     strength=1,         leadership=false,     leadershipBonus=0,
     indestructible=false, detectionRange=0,  respawns=false, respawnTurns
     maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

   { id="aragorn",       name="Aragorn, Son of Arathorn",
     class="FellowshipGuard", side="FREE_PEOPLES", start="bree",
     strength=5,         leadership=true,      leadershipBonus=1,
     indestructible=false, detectionRange=0,  respawns=false, respawnTurns
     maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

   { id="legolas",       name="Legolas Greenleaf",
     class="FellowshipGuard", side="FREE_PEOPLES", start="rivendell",
     strength=3,         leadership=false,     leadershipBonus=0,
     indestructible=false, detectionRange=0,  respawns=false, respawnTurns
     maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

   { id="gimli",         name="Gimli, Son of Gloin",
     class="FellowshipGuard", side="FREE_PEOPLES", start="rivendell",
     strength=3,         leadership=false,     leadershipBonus=0,
     indestructible=false, detectionRange=0,  respawns=false, respawnTurns
     maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

   { id="rohan-cavalry", name="Riders of Rohan",
     class="FellowshipGuard", side="FREE_PEOPLES", start="edoras",
     strength=4,         leadership=false,     leadershipBonus=0,
```

```
  indestructible=false, detectionRange=0,  respawns=false, respawnTurns
  maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

{ id="gondor-army",   name="Army of Gondor",
  class="GondorArmy", side="FREE_PEOPLES",  start="minas-tirith",
  strength=5,         leadership=false,     leadershipBonus=0,
  indestructible=false, detectionRange=0,  respawns=false, respawnTurns
  maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

{ id="gandalf",       name="Gandalf the Grey",
  class="Maia",       side="FREE_PEOPLES",  start="rivendell",
  strength=4,         leadership=false,     leadershipBonus=0,
  indestructible=false, detectionRange=0,  respawns=false, respawnTurns
  maia=true,          maiaAbilityPaths=[], ignoresFortress=false, canFo
  cooldown=3 },

# DARK SIDE — 7 units

{ id="witch-king",    name="The Witch-King of Angmar",
  class="Nazgul",     side="SHADOW",        start="minas-morgul",
  strength=5,         leadership=true,      leadershipBonus=1,
  indestructible=true, detectionRange=2,   respawns=false, respawnTurns
  maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

{ id="nazgul-2",      name="The Dark Marshal",
  class="Nazgul",     side="SHADOW",        start="minas-morgul",
  strength=3,         leadership=false,     leadershipBonus=0,
  indestructible=false, detectionRange=1,  respawns=true, respawnTurns=
  maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

{ id="nazgul-3",      name="The Betrayer",
  class="Nazgul",     side="SHADOW",        start="minas-morgul",
  strength=3,         leadership=false,     leadershipBonus=0,
  indestructible=false, detectionRange=1,  respawns=true, respawnTurns=
  maia=false,         maiaAbilityPaths=[], ignoresFortress=false, canFo

{ id="uruk-hai-legion", name="Uruk-hai Legion",
  class="UrukHaiLegion", side="SHADOW",    start="isengard",
  strength=5,         leadership=false,     leadershipBonus=0,
  indestructible=false, detectionRange=0,  respawns=false, respawnTurns
  maia=false,         maiaAbilityPaths=[], ignoresFortress=true, canFor

{ id="saruman",       name="Saruman the White",
  class="Maia",       side="SHADOW",        start="isengard",
```

```
    strength=4,          leadership=false,     leadershipBonus=0,
    indestructible=false, detectionRange=0,  respawns=false, respawnTurns
    maia=true,
    maiaAbilityPaths=["fangorn-to-isengard","helms-deep-to-isengard",
                      "fords-of-isen-to-isengard","tharbad-to-fords-of-is
                      "fords-of-isen-to-edoras"],
    ignoresFortress=false, canFortify=false, cooldown=2 },

  { id="sauron",         name="Sauron, the Dark Lord",
    class="Maia",        side="SHADOW",        start="mordor",
    strength=5,          leadership=false,     leadershipBonus=0,
    indestructible=true, detectionRange=0,   respawns=false, respawnTurns
    maia=true,           maiaAbilityPaths=[], ignoresFortress=false, canFc
    cooldown=0 }
]

hidden-until-turn = 3
max-turns = 40
turn-duration-seconds = 60
```

Total: 14 units.

3.4 Unit Roles

Unit	Str	Role
Ring Bearer	1	Move to Mount Doom. Never fights. Moves like all other units (auto-advances on assigned route).
Aragorn	5	Strongest Light Side fighter. Leadership gives +1 strength to all co-located Light Side allies in combat. Station at path endpoints to block Nazgul. Escort the Ring Bearer's route to protect endpoints ahead.
Legolas	3	Mobile escort. Contest path endpoints. Follow the Ring Bearer's route to deny Nazgul blocking positions.
Gimli	3	Mobile escort. Contest path endpoints. Fight alongside Aragorn for maximum leadership benefit.
Rohan Cavalry	4	Fast mobile unit. Guards the Fords of Isen on Route 4.
Gondor Army	5	Fortifies Minas Tirith (+2 defence). Guards the southern corridor.
Gandalf	4	Opens BLOCKED paths for 2 turns. Directly enables Ring Bearer movement when the route is blocked.

Unit	Str	Role
Witch-King	5	Primary hunter. Indestructible (strength floors at 1). Detection range 2. Leadership +1 to co-located Nazgul.
Nazgul 2 & 3	3	Block path endpoints. Raise surveillance. Respawn after 3 turns if destroyed.
Uruk-hai Legion	5	Ignores fortress terrain bonus when attacking. Needed to breach a fortified Minas Tirith alongside the Witch-King.
Saruman	4	Permanently corrupts Southern Corridor paths. Disabled when Isengard falls.
Sauron	5	Passive: all Nazgul gain +1 detection range while Sauron is in Mordor and active. Never moves.

3.5 Maia Abilities

All three Maia units have `maia=true` in config. The ability dispatched depends on which unit is sending the `MaiaAbility` order — determined by reading `config.id`, never hardcoded.

Gandalf — OpenPath:

- Requirement: Gandalf in an endpoint region of the target path; path is BLOCKED.
- Effect: Path becomes TEMPORARILY_OPEN for 2 turns. Ring Bearer and allies can cross as if OPEN.
- After 2 turns: reverts to BLOCKED (if blocker still present) or OPEN.
- Cooldown: 3 turns after use.

Saruman — CorruptPath:

- Requirement: Saruman in an endpoint region; path in `maiaAbilityPaths`; path is OPEN, THREATENED, or BLOCKED.
- Effect: Permanently sets `surveillanceLevel=3` on the path. Cannot be undone.
- Consequence: Any Ring Bearer crossing this path becomes `exposed=true` that turn regardless of Nazgul positions.
- Cooldown: 2 turns. Disabled permanently when Isengard falls to Light Side.

Sauron — Eye of Sauron (passive, no order required):

- While Sauron is in Mordor and ACTIVE: all Nazgul gain +1 effective detection range.
- Witch-King: range 2 → 3. Nazgul 2 & 3: range 1 → 2.
- Applied automatically in the detection step every turn end.

3.6 Detection Formula

Applied every turn end. Suppressed on turns 1 through `hidden-until-turn` (= 3).

```
for each Nazgul N:
  range = N.config.detectionRange
  if sauron.region == "mordor" and sauron.status == ACTIVE:
    range += 1
  if graph.distance(N.region, ringBearer.trueRegion) <= range:
    exposed = true
    emit RingBearerDetected(trueRegion)  // Dark Side only
```

---

4. Combat

4.1 Formula

```
terrain_bonus (defender's region):
  FORTRESS  → +2
  MOUNTAINS → +1
  others    → 0

fortification_bonus:
  region.fortified == true → +2
  only GondorArmy can fortify via FortifyRegion order

ignoresFortress (attacker config.ignoresFortress == true):
  terrain_bonus NOT added to defender power
  fortification_bonus still applies normally

leadership_bonus:
  each ally co-located with a leader (same side, same region, not the
leader itself)
  receives +leader.config.leadershipBonus to effective strength

indestructible (config.indestructible == true):
  newStrength = max(1, currentStrength - damage)
  never DESTROYED, never RESPAWNING

attacker_power = sum of attackers' effective strengths
defender_power = sum of defenders' effective strengths
               + terrain_bonus  (skipped if ignoresFortress)
               + fortification_bonus
```

```
if attacker_power > defender_power:
  damage = attacker_power - defender_power
  region control → attacker's side
else:
  each attacker loses 1 strength
  region control unchanged
```

4.2 Combat Examples

UrukHai alone vs fortified GondorArmy at Minas Tirith:

```
attacker_power = 5  (ignoresFortress: terrain skipped)
defender_power = 5 + 0 + 2 = 7  (fortification applies)
Result: 5 vs 7 → GondorArmy holds.
```

Fortification makes GondorArmy safe from UrukHai alone.

Witch-King + UrukHai vs fortified GondorArmy:

```
UrukHai effective strength = 5 + 1 (Witch-King leadership) = 6
attacker_power = 5 + 6 = 11
defender_power = 5 + 0 + 2 = 7
Result: 11 vs 7 → Dark Side wins. Minas Tirith falls.
```

The combined attack breaks through. The Dark Side must commit both units.

Fellowship group vs UrukHai at Isengard:

```
Gimli effective strength   = 3 + 1 (Aragorn leadership) = 4
Legolas effective strength = 3 + 1 (Aragorn leadership) = 4
attacker_power = 5 + 4 + 4 = 13
defender_power = 5 + 2 = 7  (UrukHai defending FORTRESS, ignoresFortress
only applies when attacking)
Result: 13 vs 7 → Fellowship wins. Isengard falls, Saruman disabled.
```

Aragorn alone vs UrukHai at Isengard:

```
attacker_power = 5
defender_power = 5 + 2 = 7
Result: 5 vs 7 → Aragorn repelled.
```

No single unit wins against a fortress-defended army. The group is required.

---

5. Orders

5.1 Rules

- Maximum one order per unit per turn. A second order for the same unit: `DUPLICATE_UNIT_ORDER`.
- Wrong turn number: `WRONG_TURN`.
- The UI shows only legal orders when a player clicks a unit via `GET /orders/available?unitId=X&playerId=Y`.

5.2 Movement

All units including the Ring Bearer move via the same mechanism:

1. Submit `AssignRoute` with an ordered list of path IDs.
2. The unit auto-advances one step per turn along its route.
3. To stop, redirect, or change direction: submit `RedirectUnit`.

The Ring Bearer's `currentRegion` is always `""` in public state. Only `RingBearerActor` (Option A) or `RingBearerKTable` (Option B) holds the true region — never exposed to shared topics.

5.3 Order Types

AssignRoute

```
{"orderType":"ASSIGN_ROUTE","playerId":"str","unitId":"str","turn":0,
 "pathIds":["pathId1","pathId2"]}
```

RedirectUnit

```
{"orderType":"REDIRECT_UNIT","playerId":"str","unitId":"str","turn":0,
 "newPathIds":["pathId1","pathId2"]}
```

DestroyRing — win the game

```
{"orderType":"DESTROY_RING","playerId":"str","unitId":"ring-bearer","turr
```

Valid only when Ring Bearer is at `mount-doom` and no Dark Side unit is there.

MaiaAbility — Gandalf OpenPath or Saruman CorruptPath

```
{"orderType":"MAIA_ABILITY","playerId":"str","unitId":"str","turn":0,"tar
```

BlockPath

```
{"orderType":"BLOCK_PATH","playerId":"str","unitId":"str","turn":0,"pathI
```

Unit must be in one of the path's endpoint regions.

SearchPath (Dark Side only)

```
{"orderType":"SEARCH_PATH","playerId":"str","unitId":"str","turn":0,"path
```

Raises `surveillanceLevel` by 1 (max 3).

AttackRegion

```
{"orderType":"ATTACK_REGION","playerId":"str","unitId":"str","turn":0,"ta
```

ReinforceRegion

```
{"orderType":"REINFORCE_REGION","playerId":"str","unitId":"str","turn":0,
```

FortifyRegion (GondorArmy only)

```
{"orderType":"FORTIFY_REGION","playerId":"str","unitId":"gondor-army","tu
```

DeployNazgul (Dark Side only)

```
{"orderType":"DEPLOY_NAZGUL","playerId":"str","unitId":"str","turn":0,"ta
```

5.4 Error Codes

`WRONG_TURN` `NOT_YOUR_UNIT` `INVALID_PATH` `PATH_BLOCKED`
`UNIT_NOT_ADJACENT` `INVALID_TARGET` `DUPLICATE_UNIT_ORDER`
`ABILITY_ON_COOLDOWN` `MAIA_DISABLED` `DESTROY_CONDITION_NOT_MET`

---

6. Turn Processing

Every turn end executes in this fixed order:

```
Step 1.  Collect all validated orders for this turn.

Step 2.  Process AssignRoute and RedirectUnit orders.

Step 3.  Process BlockPath and SearchPath orders.
         For each newly BLOCKED path: find all units with that path
         in their route → emit RouteCompromised → skip auto-advance.
         A path reverts from BLOCKED if the blocking unit is no longer
         at its endpoint at this step.

Step 4.  Process ReinforceRegion and DeployNazgul orders.

Step 5.  Process FortifyRegion. Sets fortified=true, fortifyTurns=2.

Step 6.  Process MaiaAbility orders.
         Gandalf: path → TEMPORARILY_OPEN (tempOpenTurns=2). Set cooldown.
         Saruman: path.surveillanceLevel=3. Permanent. Set cooldown.

Step 7.  Auto-advance all units with assigned routes (Ring Bearer
included).
         BLOCKED path → unit stays; RouteBlocked emitted.
         OPEN / THREATENED / TEMPORARILY_OPEN → unit moves one step.
         Route complete → RouteComplete emitted.
         If Ring Bearer advances and path.surveillanceLevel >= 1
         and turn > hidden-until-turn:
           exposed=true. Emit RingBearerSpotted to Dark Side only.
         Emit RingBearerMoved to Light Side only (every advance).

Step 8.  Process AttackRegion. Resolve combat per Section 4.
         If Isengard falls to Light Side: disable Saruman permanently.

Step 9.  Decrement TEMPORARILY_OPEN timers.
         Timer=0 and blocker present → BLOCKED.
         Timer=0 and no blocker → OPEN.

Step 10. Decrement fortification timers.
         Timer=0 → fortification expires.
```

```
Step 11. Decrement respawn and cooldown counters.
         RespawnTurns=0 → unit returns to home region at full strength.

Step 12. Run detection check (suppressed if turn <= hidden-until-turn).
         Apply detection formula from Section 3.6.

Step 13. Evaluate win conditions.
         Win or draw → emit GameOver (EOS). Game ends.
         Emit WorldStateSnapshot.
         Reset exposed=false.
```

---

7. Player Guide

7.1 Light Side

1. Assign a route to the Ring Bearer. It will auto-advance each turn.
2. Assign FellowshipGuards to follow the same route — they will occupy the path endpoints, denying Nazgul permanent blocking positions.
3. If a key path is BLOCKED, move Gandalf adjacent and use MaiaAbility to open it for 2 turns.
4. If the northern corridor is threatened, consider switching the Ring Bearer to Route 4 (Southern Corridor) via Tharbad.
5. Keep GondorArmy at Minas Tirith and fortify — UrukHai alone cannot breach it.

7.2 Dark Side

1. Assign Nazgul 2 and Nazgul 3 to chokepoints early — before Turn 4 when detection activates.
2. Use SearchPath to raise surveillance on paths the Ring Bearer is likely to use.
3. When a detection event fires, move the Witch-King there immediately.
4. Saruman should corrupt a key Route 4 path early — `fords-of-isen-to-edoras` makes the southern corridor permanently risky.
5. To take Minas Tirith, UrukHai and Witch-King must attack together.

PART 2 — COMMON: KAFKA

Required for both Option A and Option B.

---

8. Role of Kafka

Kafka is the event backbone. All orders flow through Kafka for validation before the game engine processes them. All game events flow through Kafka before reaching the browser.

This means:

- The game engine (Akka or Go) is fully decoupled from the browser.
- Order validation happens in one place regardless of which engine is used.
- The event log is durable — the system can recover from crashes by replaying Kafka topics.

The Kafka layer is identical in both options. It does not know which engine you chose.

---

9. Topics

topic	partition key	partitions	replication	cleanup	retention
game.orders.raw	playerId	3	3	delete	1h
game.orders.validated	unitId	6	3	delete	1h
game.events.unit	unitId	6	3	delete	7d
game.events.region	regionId	6	3	delete	7d
game.events.path	pathId	6	3	delete	7d
game.session	—	1	3	compact	—
game.broadcast	—	1	3	delete	1h
game.ring.position	—	1	3	delete	1h
game.ring.detection	playerId	2	3	delete	1h
game.dlq	errorCode	3	3	delete	7d

10 topics total.

`game.ring.position` — carries `RingBearerMoved`. Consumed by Light Side SSE only.
`game.ring.detection` — carries `RingBearerDetected` and `RingBearerSpotted`. Consumed by Dark Side SSE only.
`game.broadcast` — carries `WorldStateSnapshot`. Delivered to both sides; Ring Bearer position stripped before delivery to Dark Side.
`game.session` — log-compacted; always holds the latest game session state.

---

10. Avro Schemas

Register all schemas in Confluent Schema Registry. Subject naming: `{topicName}-value`.

```
OrderSubmitted:         playerId, unitId, orderType, payload(bytes), turn,
timestamp
OrderValidated:         same as OrderSubmitted + routeRiskScore(nullable
int)
UnitMoved:              unitId, from, to, turn, timestamp
PathStatusChanged:      pathId, newStatus, surveillanceLevel,
tempOpenTurns, turn, timestamp
PathCorrupted:          pathId, turn, timestamp
RegionControlChanged:   regionId, newController, turn, timestamp
BattleResolved:         regionId, attackerWon, turn, timestamp
RingBearerMoved:        trueRegion, turn, timestamp
RingBearerDetected:     regionId, turn, timestamp
RingBearerSpotted:      pathId, turn, timestamp
WorldStateSnapshot:     turn, regions[], units[], timestamp
GameOver:               winner, cause, turn, timestamp
DLQEntry:               originalTopic, partition, offset, errorCode,
                        errorMessage, rawPayload(bytes), timestamp
```

Schema evolution requirement: Add nullable field `routeRiskScore` to `OrderValidated` as V2. Deploy V2 while V1 consumers continue running without error. Demonstrate this live during the demo.

---

11. Kafka Streams Topology 1 — Order Validation

Source: `game.orders.raw`

Sinks: `game.orders.validated` (valid) and `game.dlq` (invalid)

KTables:

- `TurnKTable` — current turn, sourced from `game.session`
- `UnitKTable` — current unit states, sourced from `game.events.unit`
- `PathKTable` — current path states, sourced from `game.events.path`

8 validation rules:

#	Rule	Error Code
1	`order.turn` does not match current turn	WRONG_TURN

#	Rule	Error Code
2	Unit does not belong to submitting player's side	NOT_YOUR_UNIT
3	Ring Bearer route: next path is BLOCKED	PATH_BLOCKED
4	Ring Bearer route: path not in assigned route	INVALID_PATH
5	BlockPath / SearchPath: unit not in an endpoint region	UNIT_NOT_ADJACENT
6	AttackRegion: target not adjacent or not enemy-controlled	INVALID_TARGET
7	MaiaAbility: unit cooldown > 0	ABILITY_ON_COOLDOWN
8	Same unitId appears more than once this turn	DUPLICATE_UNIT_ORDER

---

12. Kafka Streams Topology 2 — Route Risk Enrichment

Source: `game.orders.validated` — filter `ASSIGN_ROUTE` and `REDIRECT_UNIT`

KTables: `PathKTable`, `RegionKTable`

```
routeRiskScore =
    sum(region.threatLevel       for each destination region in route)
  + sum(path.surveillanceLevel   for each path in route) * 3
  + count(THREATENED paths)  * 2
  + count(BLOCKED paths)     * 5
  + nazgulProximityCount     * 2
```

`nazgulProximityCount` = number of Nazgul within 2 graph hops of any region in the route, sourced from `UnitKTable`.

Attach `routeRiskScore`, `threatenedPaths[]`, `blockedPaths[]` to the record. Re-emit enriched record to `game.orders.validated`.

---

13. Exactly-Once Semantics

The `GameOver` event must be produced with `enable.idempotence=true`. It must appear **exactly once** in `game.broadcast` even if the game engine crashes and restarts mid-transaction. Verify with `kafka-console-consumer` during Demo Scenario 3.

PART 3A — OPTION A: AKKA

14. Architecture

```
Browser A (Light Side)                Browser B (Dark Side)
  POST /order                           POST /order
  GET  /events (SSE)                    GET  /events (SSE)
  GET  /orders/available                GET  /orders/available
       |                                     |
       +--------------+--------------------+
                      |
         +------------+------------+
         |    Akka HTTP Layer      |
         |  REST API + SSE Server  |
         +------------+------------+
                      |
            Produces → game.orders.raw
            Consumes ← game.broadcast
                       game.events.*
                       game.ring.position  (Light Side only)
                       game.ring.detection (Dark Side only)
                      |
         +------------+------------+
         |         Kafka           |
         |   (Part 2 — unchanged)  |
         +------------+------------+
                      |
            Consumes game.orders.validated
            Produces all game.events.*
                      |
         +------------+------------+
         |    Akka Game Engine     |
         |  3-node cluster         |
         |  Cluster Sharding:      |
         |    UnitActor   (14)     |
         |    RegionActor (22)     |
         |    PathActor   (35)     |
         |  Cluster Singletons:    |
         |    RingBearerActor      |
         |    WorldStateActor      |
```

```
        |     GameSessionActor     |
        +--------------------------+
```

---

15. Cluster Setup

- 3 nodes in Docker Compose. Node 1 is the seed.
- **Cluster Sharding:** `UnitActor`, `RegionActor`, `PathActor`
- **Cluster Singletons:** `RingBearerActor`, `WorldStateActor`, `GameSessionActor`
- **Persistence:** LevelDB journal and snapshots in Docker volumes. Snapshot every 10 events.

---

16. Actor Hierarchy

```
GameGuardian
  ├── GameSessionActor   (ClusterSingleton)
  ├── WorldStateActor    (ClusterSingleton)
  ├── RingBearerActor    (ClusterSingleton)
  ├── UnitSupervisor
  |     └── UnitActor   (ShardRegion — 14 instances)
  ├── RegionSupervisor
  |     └── RegionActor (ShardRegion — 22 instances)
  └── PathSupervisor
        └── PathActor   (ShardRegion — 35 instances)
```

---

17. UnitConfig

```
case class UnitConfig(
  id:               String,
  name:             String,
  unitClass:        UnitClass,
  side:             Side,
  startRegion:      String,
  strength:         Int,
  leadership:       Boolean,
  leadershipBonus:  Int,
  indestructible:   Boolean,
  detectionRange:   Int,
  respawns:         Boolean,
  respawnTurns:     Int,
```

```
  maia:              Boolean,
  maiaAbilityPaths: List[String],
  ignoresFortress:  Boolean,
  canFortify:       Boolean,
  cooldown:         Int
)
```

Loaded from shared config at startup. Stored in `Map[String, UnitConfig]`.

No unit id string literal appears anywhere in actor logic.

---

18. UnitActor

Shard key: unit id.

State:

```
case class UnitState(
  id:           String,
  region:       String,     // always "" for ring-bearer
  strength:     Int,
  status:       Status,     // ACTIVE | DESTROYED | RESPAWNING
  respawnTurns: Int,
  route:        List[String],
  routeIdx:     Int,
  cooldown:     Int
)
```

ApplyDamage — config-driven:

```
val raw    = state.strength - damage
val newStr = if (config.indestructible) math.max(1, raw) else raw
if (newStr <= 0 && !config.indestructible)
  if (config.respawns)
    state.copy(strength=0, status=RESPAWNING,
               respawnTurns=config.respawnTurns, region="")
  else
    state.copy(strength=0, status=DESTROYED)
else state.copy(strength=newStr)
```

State machine:

```
ACTIVE     → ApplyDamage (str > 0)           → ACTIVE
ACTIVE     → ApplyDamage (0, indestructible) → ACTIVE (str=1)
ACTIVE     → ApplyDamage (0, respawns=true)  → RESPAWNING
ACTIVE     → ApplyDamage (0, others)         → DESTROYED
RESPAWNING → respawnTurns=0                  → ACTIVE (home, full
strength)
RESPAWNING → any other command               → rejected
DESTROYED  → any command                     → rejected
```

Persisted events: `UnitMoved` `RouteAssigned` `RouteRedirected` `UnitDamaged` `UnitDestroyed` `UnitRespawned` `CooldownSet`

---

19. PathActor

Shard key: path id.

PathStatus: `OPEN | THREATENED | BLOCKED | TEMPORARILY_OPEN`

State machine:

```
OPEN / THREATENED → BlockPath              → BLOCKED
OPEN              → ThreatPath             → THREATENED
THREATENED        → ClearPath              → OPEN
BLOCKED           → ClearPath              → OPEN
BLOCKED           → MaiaAbility (Gandalf)  → TEMPORARILY_OPEN (timer=2)
TEMPORARILY_OPEN  → timer=0, blocker       → BLOCKED
TEMPORARILY_OPEN  → timer=0, no blocker    → OPEN
Any               → SearchPath             → surveillanceLevel += 1 (max
3)
Any               → MaiaAbility (Saruman)  → surveillanceLevel=3
(permanent)
```

Persisted events: `PathStatusChanged` `SurveillanceLevelChanged` `PathCorrupted`

---

20. RegionActor

Shard key: region id. 22 instances.

Tracks: `controlledBy`, `threatLevel`, `fortified`, `fortifyTurns`, `unitsPresent`.

Persisted events: `RegionControlChanged` `RegionFortified` `FortificationExpired` `BattleResolved` `IsengardDestroyed`

`IsengardDestroyed` is emitted when Isengard's control changes to FREE_PEOPLES. `WorldStateActor` listens and permanently disables Saruman.

---

21. RingBearerActor

Cluster Singleton. Owns `trueRegion`. Never emits it to any shared topic.

```
case class RingBearerState(
  trueRegion:         String,
  exposed:            Boolean,
  route:              List[String],
  routeIdx:           Int,
  lastDetectedTurn:   Option[Int],
  lastDetectedRegion: Option[String]
)
```

---

22. WorldStateActor and GameSessionActor

WorldStateActor (Cluster Singleton): drives the 13-step turn processing from Section 6. Emits `WorldStateSnapshot` to `game.broadcast` each turn.

GameSessionActor (Cluster Singleton): fires `TurnEnded` every `turn-duration-seconds`. Evaluates win/draw. Emits `GameOver` with EOS guarantee.

22.1 Analysis (Option A)

The game engine must provide route risk and interception analysis to the browser via two endpoints. Implement these using Akka's native mechanisms — the approach is your architectural decision.

Acceptable implementations:

- `WorldStateActor` computes both analyses at each turn end and includes results in `WorldStateSnapshot`
- Dedicated `AnalysisActor` s receive queries and respond with results
- Akka Streams for concurrent fan-out/fan-in computation

Regardless of approach, these two endpoints must work:

- `GET /analysis/routes` — ranked route list with risk scores (Light Side only)
- `GET /analysis/intercept` — interception plan per Nazgul (Dark Side only)

Route risk formula (same as Topology 2):
riskScore =
sum(region.threatLevel for each destination region)
+ sum(path.surveillanceLevel for each path) * 3
+ count(BLOCKED paths) * 5
+ count(THREATENED paths) * 2
+ nazgulProximityCount * 2

Interception score per (Nazgul, route-candidate) pair:
interceptWindow = rbTurnsToReach - turnsToIntercept
score = interceptWindow >= 0 ?
1.0 - (turnsToIntercept / routeLength) : 0.0

Document your chosen approach and justify it in the architecture document — this is one of the key places where Option A and Option B differ architecturally.

---

23. Supervision

```
UnitSupervisor → UnitActor:
  Exponential backoff 200ms-30s. maxRestarts=5 per 60s, then escalate.

RegionSupervisor → RegionActor:
  Resume on IllegalOrderException. Backoff restart on others.

PathSupervisor → PathActor:
  Resume on IllegalTransitionException. Backoff restart on others.
```

```
GameGuardian → all:
  Stop on escalation. Emit to game.dlq.
```

---

24. Required Unit Tests (Option A)

Run with `sbt test`. No Docker or Kafka required.

UnitActorSpec — 10 cases:

1. AssignRoute → route updated
2. Empty route + auto-advance → RouteRejected
3. OPEN path + auto-advance → unit moves
4. BLOCKED path + auto-advance → RouteBlocked, region unchanged
5. Last path + auto-advance → RouteComplete
6. ApplyDamage (str > 0) → strength reduced
7. ApplyDamage to 0, respawns=true → RESPAWNING
8. ApplyDamage past 0, indestructible=true → strength=1, ACTIVE
9. RESPAWNING + any command → rejected
10. DESTROYED + any command → rejected

PathActorSpec — 7 cases:

1. OPEN + ThreatPath → THREATENED
2. THREATENED + BlockPath → BLOCKED
3. BLOCKED + MaiaAbility (Gandalf) → TEMPORARILY_OPEN, timer=2
4. TEMPORARILY_OPEN + timer=0, blocker present → BLOCKED
5. BLOCKED + ClearPath → OPEN
6. Any + SearchPath → surveillanceLevel increases (max 3)
7. Any + MaiaAbility (Saruman) → surveillanceLevel=3, PathCorrupted emitted

CombatSpec — 8 cases:

1. Attacker(5) vs Defender(5, PLAINS) → tie, attacker repelled
2. Attacker(5) vs Defender(5, FORTRESS) → defender wins (5 vs 7)
3. UrukHai(5, ignoresFortress) vs Defender(5, FORTRESS) → tie (5 vs 5)
4. UrukHai(5) vs Defender(5, FORTRESS, fortified) → defender wins (5 vs 7)
5. Aragorn(5, leader+1) + Gimli(3) attack → Gimli effective=4; 5+4=9 vs 5
6. Indestructible takes fatal damage → strength=1, ACTIVE
7. Nazgul (respawns) takes fatal damage → RESPAWNING
8. Non-respawning unit takes fatal damage → DESTROYED

RingBearerActorSpec — 4 cases:

1. Nazgul (range=1) at 1 hop → exposed=true
2. Nazgul (range=1) at 2 hops → exposed=false
3. Nazgul (range=2) at 2 hops → exposed=true
4. Sauron active in Mordor + Nazgul (range=1) at 2 hops → effective range=2 → exposed=true

---

PART 3B — OPTION B: GO

---

25. Architecture

```
Browser A (Light Side)                Browser B (Dark Side)
  POST /order                           POST /order
  GET  /events (SSE)                    GET  /events (SSE)
  GET  /orders/available                GET  /orders/available
       |                                     |
       +--------------+--------------------+
                      |
         +------------+------------+
         |   Go HTTP Layer         |
         |  REST API + SSE Server  |
         +------------+------------+
                      |
            Produces → game.orders.raw
            Consumes ← game.broadcast
                       game.events.*
                       game.ring.position  (Light Side only)
                       game.ring.detection (Dark Side only)
                      |
         +------------+------------+
         |         Kafka           |
         |   (Part 2 — unchanged)  |
         |   + KTable state stores |
         +------------+------------+
                      |
            Consumes game.orders.validated
            Produces all game.events.*
```

---

26. Distributed Cluster (Option B)

In Option B, the system runs as **3 Go instances** behind a load balancer. Each instance is stateless — all authoritative game state lives in Kafka KTable state stores.

State stores:

KTable	Key	Value
UnitKTable	unitId	UnitSnapshot
RegionKTable	regionId	RegionState
PathKTable	pathId	PathState
RingBearerKTable	"ring-bearer"	RingBearerState (trueRegion never exposed)

These KTables are maintained by the Kafka Streams topologies. Each Streams instance handles a subset of partitions.

How fault tolerance works:

- The 3 Go instances form a single Kafka **consumer group**.
- Each partition is assigned to exactly one instance.
- If `go-2` crashes: Kafka detects the failure, triggers consumer group rebalance, and reassigns `go-2`'s partitions to `go-1` and `go-3`.
- When `go-2` restarts: it replays its assigned partitions from Kafka and rebuilds its local KTable view.
- The game continues uninterrupted.

This is the Option B equivalent of Akka's cluster shard rebalancing. The mechanism is different — Kafka consumer group protocol instead of Akka Cluster — but the outcome is the same: state survives node failure.

Demo Scenario 3 (Option B):
Run `docker stop go-2`. Observe consumer group rebalance in Kafka logs. Confirm the game continues on `go-1` and `go-3`. Run `docker start go-2`. Observe `go-2` rejoin the consumer group and recover state from Kafka.

This architecture illustrates a fundamental principle of Go + Kafka distributed systems: **the application tier is stateless, the messaging tier holds the state.** Go instances are interchangeable — any instance can handle any request because all authoritative state lives in Kafka. Fault tolerance is delegated entirely to Kafka's consumer group protocol rather than implemented in the application layer.

This contrasts directly with Option A, where state lives inside Akka actors and fault tolerance is managed by the cluster sharding framework. Neither approach is superior — they reflect different philosophies: Option A keeps state close to logic (actors own their state), Option B separates state from logic entirely (stateless processes, stateful broker). Your architecture document must discuss this tradeoff.

---

27. UnitConfig

```
type UnitConfig struct {
    ID               string
    Name             string
    Class            string
    Side             string
    StartRegion      string
    Strength         int
    Leadership       bool
    LeadershipBonus  int
    Indestructible   bool
    DetectionRange   int
    Respawns         bool
    RespawnTurns     int
    Maia             bool
    MaiaAbilityPaths []string
    IgnoresFortress  bool
    CanFortify       bool
    Cooldown         int
}
```

Loaded from shared config at startup. Stored in `map[string]UnitConfig`.

No unit id string literal appears anywhere in game logic.

---

28. Goroutine Architecture

```
main
  ├── KafkaConsumer goroutines     one per subscribed topic
  │     events → eventCh (buffered, cap=100)
  │
  ├── EventRouter goroutine
  │     reads eventCh
```

```
│     → lightSideSSECh    ring.position + broadcast (unstripped)
│     → darkSideSSECh     ring.detection + broadcast (RB stripped)
│     → cacheUpdateCh     world state updates
│     → engineCh          validated orders
│
├── CacheManager goroutine
│     owns WorldStateCache
│     sends value copies to workers — never pointers
│
├── TurnProcessor goroutine
│     reads engineCh
│     executes 13-step turn processing
│     produces events → KafkaProducer
│
├── Go Pipeline 1 — Route Risk (Light Side)
│     Dispatcher → buffered ch (cap=20) → 4 workers
│     → unbuffered ch → Aggregator → Deliverer
│
├── Go Pipeline 2 — Interception (Dark Side)
│     Dispatcher → buffered ch (cap=30) → 4 workers
│     → unbuffered ch → Aggregator → Deliverer
│
├── SSE goroutines            one per connected player
│
└── HTTP server goroutine
```

---

29. WorldStateCache

```go
type WorldStateCache struct {
    Turn        int
    Units       map[string]UnitSnapshot
    Regions     map[string]RegionState
    Paths       map[string]PathState
    UnitConfigs map[string]UnitConfig  // read-only after startup
    LightView   LightSideView
    DarkView    DarkSideView
}

type LightSideView struct {
    RingBearerRegion string
    AssignedRoute    []string
```

```
    RouteIdx         int
}

type DarkSideView struct {
    RingBearerRegion   string  // ALWAYS "" — no code path ever sets this
    LastDetectedRegion string
    LastDetectedTurn   int
}
```

---

30. EventRouter — Information Hiding

The EventRouter is the single enforcement point for information asymmetry.

```
switch event.Topic {

case "game.ring.position":
    lightSideSSECh <- event
    // never darkSideSSECh

case "game.ring.detection":
    darkSideSSECh <- event
    // never lightSideSSECh

case "game.broadcast":
    lightSideSSECh <- event
    darkSideSSECh  <- stripRingBearer(event)
    // stripRingBearer sets ring-bearer.currentRegion = ""

case "game.events.unit",
     "game.events.region",
     "game.events.path":
    lightSideSSECh <- event
    darkSideSSECh  <- event
}
```

`DarkView.RingBearerRegion` is always `""`. This is enforced here and tested with `go test -race`.

---

31. Select Loop

```
for {
    select {
    case msg  := <-kafkaConsumerCh:
    case conn := <-newConnectionCh:
    case disc := <-disconnectCh:
    case req  := <-analysisRequestCh:
    case snap := <-cacheUpdateCh:
    case tick := <-time.After(60 * time.Second):
    case sig  := <-signalCh:
    }
}
```

All 7 cases must be handled. Verify zero goroutine leaks with `pprof` after 10 turns.

---

32. Go Pipeline 1 — Route Risk (Light Side)

4 workers. Buffer cap 20. Trigger: `GET /analysis/routes` or `RouteCompromised` event.

Each worker receives a value copy of the cache and computes:

```
riskScore =
    sum(region.threatLevel for each destination region)
  + sum(path.surveillanceLevel for each path) * 3
  + count(BLOCKED paths)     * 5
  + count(THREATENED paths)  * 2
  + nazgulProximityCount     * 2
```

`nazgulProximityCount` = Nazgul within 2 graph hops of any region in the route.

Output: `RankedRouteList{routes[], recommended, warnings[]}`.

Cancellation: `context.Context` + or-done pattern.
Shutdown: `sync.WaitGroup` at every stage boundary.
Timeout: 2 seconds → return partial result.

---

33. Go Pipeline 2 — Interception (Dark Side)

4 workers. Buffer cap 30. Trigger: `GET /analysis/intercept` or `RingBearerDetected` event.

Each worker processes one (Nazgul, route-candidate) pair:

```
turnsToIntercept = graph.shortestPath(nazgul.region, routeRegion)
rbTurnsToReach   = sum of traversal costs to that region
interceptWindow  = rbTurnsToReach - turnsToIntercept
score = interceptWindow >= 0 ?
        1.0 - (turnsToIntercept / routeLength) : 0.0
```

Output: `InterceptPlan{byUnit[{unitId, targetRegion, score}]}`.

---

34. HTTP API

Endpoint	Method	Description
/game/start	POST	`{"mode":"HVH"}`
/order	POST	Publish to game.orders.raw; return 202
/game/state	GET	World state; ring-bearer region stripped for Dark Side
/orders/available	GET	`?unitId=X&playerId=Y`
/analysis/routes	GET	Pipeline 1 result (Light Side only)
/analysis/intercept	GET	Pipeline 2 result (Dark Side only)
/events	GET	SSE stream
/health	GET	200 or 503

---

35. Required Unit Tests (Option B)

Run with `go test ./...`. No Docker or Kafka required.

combat_test.go — 6 cases:

1. Attacker(5) vs Defender(5, PLAINS) → tie
2. Attacker(5) vs Defender(5, FORTRESS) → defender wins
3. UrukHai(5, ignoresFortress) vs Defender(5, FORTRESS) → tie
4. UrukHai(5) vs Defender(5, FORTRESS, fortified) → defender wins
5. Leadership bonus applied correctly to co-located allies
6. Indestructible unit: strength floors at 1

router_test.go — 3 cases (all with `go test -race`):

1. WorldStateSnapshot with ring-bearer region set → Dark Side receives `currentRegion=""`, Light Side receives real value
2. `RingBearerMoved` event → never reaches Dark Side SSE channel
3. `cache.DarkView.RingBearerRegion` is always `""` after any cache update

pipeline1_test.go — 2 cases:

1. Route with known threat and surveillance values → correct riskScore computed
2. Nazgul within 2 hops → proximity count adds correctly to score

pipeline2_test.go — 2 cases:

1. Positive intercept window → score > 0
2. Negative intercept window → score = 0.0

PART 4 — DELIVERABLES AND ASSESSMENT

---

36. Repository Structure

```
ring-of-the-middle-earth/
├── docker-compose.yml
├── Makefile
├── README.md            ← state your technology choice here
├── config/
│   ├── units.conf       ← shared unit config (Section 3.3)
│   └── map.conf         ← 22 regions + 35 paths
├── kafka/
│   ├── streams/         ← Topologies 1 and 2
│   └── schemas/         ← all .avsc files
├── option-a/            ← if you chose Akka
│   ├── build.sbt
│   └── src/
│       ├── main/scala/rotr/
│       └── test/scala/rotr/
├── option-b/            ← if you chose Go
│   ├── go.mod
│   ├── internal/
│   └── tests/
└── ui/
```

```
    ├── index.html
    ├── game.js
    └── style.css
```

`make up` must start the entire system.

`make test` must run all unit tests without Docker.

---

37. Technology Versions

Component	Version
Kafka	3.6+
Confluent Schema Registry	7.x
Scala (Option A)	2.13 or 3.x
Akka Typed (Option A)	2.8.x
Akka Persistence (Option A)	2.8.x (LevelDB)
Alpakka Kafka (Option A)	6.x
Go (Option B)	1.22+
confluent-kafka-go (Option B)	2.x
UI	Vanilla JS + SSE. No React / Vue / Angular.

---

38. Grading Rubric — 100 Points

Kafka — 30 points (common to both options)

#	Criterion	Pts	Evidence
K1	10 topics with correct partition / replication / cleanup config	3	`kafka-topics.sh --describe`
K2	All Avro schemas registered in Schema Registry	4	Schema Registry UI
K3	Schema evolution: V2 deployed while V1 consumers run without errors	4	Live during demo
K4	Topology 1: all 8 rules produce correct error codes	10	One invalid order per rule
K5	Topology 2: correct routeRiskScore on enriched records	4	Inspect record in game.orders.validated

#	Criterion	Pts	Evidence
K6	GameOver appears exactly once after engine crash	5	Demo Scenario 3

Option A — Akka — 70 points

#	Criterion	Pts	Evidence
A1	3-node cluster: kill a node, observe rebalance, state recovers	8	Demo Scenario 3
A2	No unit id hardcoding in actor logic	8	Code review in Q&A
A3	UnitActor state machine: all transitions correct	5	UnitActorSpec — 10 cases
A4	PathActor: all 4 statuses and transitions correct	5	PathActorSpec — 7 cases
A5	Combat formula: all modifiers correct	7	CombatSpec — 8 cases
A6	Detection formula + Sauron amplifier + hidden-until-turn	5	RingBearerActorSpec — 4 cases
A7	Maia dispatch: same order type → different effect by config	5	Demo Scenario 2
A8	Path blocking: reverts when blocking unit leaves endpoint	5	Demo Scenario 2
A8b	Analysis endpoints: /analysis/routes and /analysis/intercept return correct results	5	Demo — both browsers show analysis panels
A9	Information hiding: Dark Side never receives Ring Bearer position	7	Demo Scenario 1
A10	Full HVH game playable end-to-end	5	Demo Scenario 1
A11	Architecture document	10	Submitted with repo

Option B — Go — 70 points

#	Criterion	Pts	Evidence
B1	No unit id hardcoding in game logic	8	Code review in Q&A
B2	3 Go instances + consumer group rebalance + KTable state recovery	8	Demo Scenario 3
B3	Combat formula: all modifiers correct; combat_test.go passes	7	`go test`
B4	Detection formula + Sauron amplifier + hidden-until-turn	5	Gameplay during demo

#	Criterion	Pts	Evidence
B5	Maia dispatch: same order type → different effect by config	5	Demo Scenario 2
B6	Path blocking: reverts when blocking unit leaves endpoint	5	Demo Scenario 2
B7	EventRouter: DarkView.RingBearerRegion always ""; router_test.go -race	8	`go test -race`
B8	Pipeline 1 and 2: correct output; tests pass	7	`go test` + demo panels
B9	Select loop: all 7 cases handled; no goroutine leaks	5	pprof after 10 turns
B10	Full HVH game playable end-to-end	7	Demo Scenario 1
B11	Architecture document	5	Submitted with repo

---

39. Architecture Document

Required for both options. Submit as PDF in repository root.

Required contents:

1. **System diagram** — all running services, how they connect, data flow direction.
2. **Technology-specific diagram:**
   - Option A: Actor hierarchy with supervision strategies. UnitActor and PathActor state machine diagrams with all states and transitions labelled.
   - Option B: Goroutine map — every goroutine, input/output channels, buffer capacities, termination condition.
3. **Kafka diagram** — all 10 topics, which service produces and consumes each, partition key rationale.
4. **Paradigm justification** — answer all three:
   - Why is your chosen paradigm well-suited to this problem?
   - What is genuinely harder with your chosen paradigm than it would be with the other?
   - How would the other paradigm solve the two hardest parts of your implementation?
5. **Reflection** — minimum 300 words. What was harder than expected? What would you design differently?

---

40. Demo — Three Scenarios

15-minute demo followed by 5-minute Q&A. The instructor controls all game inputs. Pre-recorded outputs are not accepted.

Scenario 1 — Information Hiding (5 min)

Instructor moves Ring Bearer to `weathertop`. Instructor moves Witch-King to `bree` (1 hop away, detection range 2). Both browsers shown side by side after turn end.

Expected observations:

- Dark Side browser receives `RING_BEARER_DETECTED`
- Light Side browser does NOT receive it
- `GET /game/state` for Dark Side returns `ring-bearer.currentRegion=""`

Scenario 2 — Maia Dispatch and Path Mechanics (5 min)

Instructor submits `MaiaAbility` for Gandalf on a BLOCKED path → path turns TEMPORARILY_OPEN (blue on map). Instructor submits the same `MaiaAbility` order type for Saruman on `fords-of-isen-to-edoras` → `PathCorrupted` fires, permanent change. After 2 turns, Gandalf's path reverts. Instructor moves a FellowshipGuard to a path endpoint and attempts a Nazgul block — block fails while the guard is present.

Scenario 3 — Fault Tolerance and Exactly-Once (5 min)

Option A: `docker stop akka-node-2` during turn processing. Class observes shard rebalance. Game continues. Node restarts and rejoins.

Option B: `docker stop go-2` during turn processing. Class observes Kafka consumer group rebalance in logs. `go-1` and `go-3` continue. `docker start go-2` — go-2 recovers state from Kafka and rejoins.

Both options: Advance Ring Bearer to Mount Doom, submit `DestroyRing`, kill the engine immediately. After restart: `kafka-console-consumer --topic game.broadcast` shows `GameOver` exactly once.

---

41. Q&A Questions

Any team member may be asked any of these.

1. Show where a Nazgul's detection range is applied in your code. There must be no string like `"witch-king"` in that logic.
2. Gandalf and Saruman both receive `MaiaAbility`. Show exactly where the dispatch happens and what config field determines the outcome.

3. A FellowshipGuard is at Lothlórien. A Nazgul tries to permanently block `lothlorien-to-emyn-muil` . Walk through exactly what happens.
4. Show in the code where the Ring Bearer's position is removed from the response before it reaches the Dark Side.
5. Sauron never receives orders. How does his Eye of Sauron effect get applied, and where in the code?
6. *(Option A)* An Akka node crashes during `WorldStateActor` turn processing. Walk through state recovery from crash to resumption.
7. *(Option B)* A Go instance crashes mid-turn. Walk through how state is recovered from Kafka on restart.
8. `game.session` uses log compaction. `game.broadcast` uses delete with 1-hour retention. If your service restarts 30 minutes into the game, how does it recover the current turn number and world state?

---

42. Academic Integrity

Use AI tools to understand concepts. Do not use them to generate complete subsystems.

This project is resistant to generation because:

- Both options require zero unit id string literals in game logic. A generated implementation writes `if unitId == "witch-king"` . The correct code reads `if config.Indestructible` . Q&A question 1 checks this live.
- Gandalf and Saruman receive the same `MaiaAbility` order type. Generated code creates two separate order types. Q&A question 2 and Demo Scenario 2 both verify single-type dispatch live.
- The Southern Corridor (Tharbad → Fords of Isen → Edoras) is defined in this document. Pipeline 1 and Pipeline 2 must score Route 4 correctly.
- `DarkView.RingBearerRegion = ""` must hold throughout. `router_test.go -race` and Demo Scenario 1 both verify this.

Required appendix in architecture document: LLM usage log. For each interaction: the prompt, what you used, what you changed or rejected. Graded for honesty.